

Atividade Prática - Integração Python API + Flutter

Como vocês viram na **Aula 19 da professora Janaína**, foi desenvolvido um projeto em Python capaz de realizar **web scraping de voos** e disponibilizar essas informações por meio de uma API.

Agora a ideia é utilizar essa API em um aplicativo mobile.

Vocês deverão desenvolver um **aplicativo em Flutter** que consuma os dados disponibilizados pelo projeto Python e apresente essas informações de forma organizada para o usuário.

O código Python será disponibilizado junto com a atividade.

Objetivo

Construir um aplicativo Flutter que se comunique com a API desenvolvida em Python e organize o projeto utilizando:

```
None
lib/
|
├─ main.dart
|
├─ pages/
|
├─ providers/
|
└─ services/
```

O Python será utilizado apenas como **backend/API**.

Toda a parte visual deverá ser desenvolvida em Flutter.

Antes de começar

Para realizar esta atividade, será necessário manter **dois projetos rodando ao mesmo tempo**:

None

1º projeto → Python / Flask

2º projeto → Flutter

Por isso, vocês deverão trabalhar com **dois terminais abertos**.

Terminal 1 - Python

No primeiro terminal, entre na pasta do projeto Python.

Instale as dependências:

Shell

```
pip install -r requirements.txt
```

Depois execute:

Shell

```
python app.py
```

O terminal deverá mostrar algo semelhante a:

None

```
Running on http://127.0.0.1:5001
```

Esse terminal deverá permanecer aberto durante a execução do aplicativo Flutter.

Para testar se a API está funcionando, abra no navegador:

None

```
http://127.0.0.1:5001/api
```

Também é possível testar diretamente a rota de voos:

None

```
http://127.0.0.1:5001/api/voos?aeroporto=SBGR&tipo=chegadas
```

Se aparecer um JSON no navegador, significa que a API está funcionando corretamente.

Terminal 2 - Flutter

Abra um segundo terminal dentro do projeto Flutter.

Caso ainda não tenha instalado as dependências necessárias, execute:

Shell

```
flutter pub add http  
flutter pub add provider  
flutter pub add device_preview
```

Depois execute:

Shell

```
flutter pub get
```

E então:

Shell

```
flutter run
```

Durante a atividade, os dois terminais deverão permanecer abertos:

None

```
TERMINAL 1  
python app.py
```

```
TERMINAL 2  
flutter run
```

O Flutter dependerá da API Python para obter os dados.

Preparando o projeto Flutter

Antes de começar a desenvolver as telas, será necessário instalar alguns pacotes no projeto Flutter.

Abra o **Terminal 2**, dentro da pasta do projeto Flutter, e execute:

```
None  
flutter pub add http  
flutter pub add provider  
flutter pub add device_preview
```

Depois execute:

```
None  
flutter pub get
```

Esses pacotes serão utilizados durante a atividade.

Parte 1 - Service

Crie a pasta:

```
None  
lib/services/
```

Dentro dela, crie um arquivo responsável pela comunicação com a API.

Exemplo:

```
None  
lib/services/voo_service.dart
```

A Service deverá:

- realizar a requisição HTTP;
- acessar a rota da API Python;
- receber os dados retornados;
- converter o JSON recebido;
- devolver os dados para o Provider.

A requisição HTTP não deverá ser feita diretamente dentro das telas.

O endereço utilizado no Flutter Web será semelhante a:

None

```
http://127.0.0.1:5001/api/voos
```

A rota poderá receber parâmetros como:

None

```
?aeroporto=SBGR&tipo=chegadas
```

Exemplo completo:

None

```
http://127.0.0.1:5001/api/voos?aeroporto=SBGR&tipo=chegadas
```

Parte 2 - Provider

Crie a pasta:

None

```
lib/providers/
```

Dentro dela, crie um Provider responsável pelos dados dos voos.

Exemplo:

None

```
lib/providers/voo_provider.dart
```

O Provider deverá utilizar:

None

```
extends ChangeNotifier
```

Ele deverá ser responsável por:

- chamar a Service;
- armazenar os voos recebidos;

- controlar o carregamento;
- armazenar uma possível mensagem de erro;
- avisar as telas quando houver alteração nos dados.

Utilize:

```
None  
notifyListeners();
```

sempre que for necessário atualizar as telas.

O fluxo esperado será:

```
None  
Page  
  ↓  
Provider  
  ↓  
Service  
  ↓  
API Python
```

E quando os dados retornarem:

```
None  
API Python  
  ↓  
Service  
  ↓  
Provider  
  ↓  
notifyListeners()  
  ↓  
Page
```

Parte 3 - Pages

Crie a pasta:

None

`lib/pages/`

O aplicativo deverá possuir pelo menos duas telas.

Tela inicial

A tela inicial deverá apresentar os voos recebidos da API.

Vocês deverão pensar em uma forma organizada de apresentar essas informações.

O aplicativo poderá mostrar, por exemplo:

- identificação do voo;
- origem;
- destino;
- horário;
- status;
- companhia aérea;
- tipo do voo.

Não é obrigatório utilizar exatamente esses campos.

Utilizem as informações disponíveis na API.

A tela deverá apresentar:

- indicador de carregamento enquanto os dados estiverem sendo buscados;
- mensagem de erro caso a API não responda;
- lista com os voos encontrados.

Ao tocar em um voo, o usuário deverá acessar uma tela de detalhes.

Tela de detalhes

Crie uma segunda tela para apresentar mais informações sobre o voo selecionado.

Utilize:

None

`Navigator.push()`

para realizar a navegação.

A tela poderá apresentar todas as informações disponíveis daquele voo.

Parte 4 - Busca por aeroporto

O usuário deverá conseguir informar o código ICAO de um aeroporto.

Exemplo:

None
SBGR

Após informar o código, o aplicativo deverá solicitar novamente os dados para a API.

Exemplos de códigos ICAO:

None
SBGR → Aeroporto de Guarulhos

SBSP → Aeroporto de Congonhas

SBBR → Aeroporto de Brasília

SBCF → Aeroporto de Confins

O código informado deverá ser enviado para a API.

Parte 5 - Tipo de voo

O aplicativo deverá permitir escolher quais voos serão exibidos.

As opções disponíveis na API são:

None
chegadas
partidas
todos

Por exemplo:

None

```
/api/voos?aeroporto=SBGR&tipo=chegadas
```

ou:

None

```
/api/voos?aeroporto=SBGR&tipo=partidas
```

Estrutura esperada

Ao final da atividade, o projeto Flutter deverá possuir uma estrutura semelhante a:

None

```
lib/  
|  
├─ main.dart  
|  
├─ pages/  
|   ├─ home_page.dart  
|   └─ detalhes_page.dart  
|  
├─ providers/  
|   └─ voo_provider.dart  
|  
└─ services/  
    └─ voo_service.dart
```

Os nomes dos arquivos podem ser diferentes, desde que a separação de responsabilidades seja mantida.

Responsabilidade de cada pasta

services

Responsável por:

None

Flutter

↓

HTTP

↓

API Python

A Service sabe **como buscar os dados**.

providers

Responsável por:

- armazenar os dados;
- controlar o estado;
- controlar carregamento;
- controlar erros;
- atualizar as telas.

O Provider sabe **quais dados o aplicativo possui**.

pages

Responsável pela interface.

As Pages sabem **como mostrar os dados**.

Importante

Não faça a requisição HTTP diretamente na tela.

Evite:

None

```
http.get(...)
```

dentro da HomePage.

A requisição deverá ficar dentro da pasta:

```
None
services/
```

A tela deverá acessar os dados por meio do:

```
None
Provider
```

Funcionamento esperado

O fluxo completo da aplicação deverá ser:

```
None
FLIGHTAWARE
  ↓
WEB SCRAPING
  ↓
PYTHON / FLASK
  ↓
API JSON
  ↓
FLUTTER
  ↓
SERVICE
  ↓
PROVIDER
  ↓
PAGES
  ↓
USUÁRIO
```

Testando o projeto

Antes de testar o Flutter, confirme que o Python está rodando.

Terminal 1

Shell

```
python app.py
```

Teste no navegador:

None

```
http://127.0.0.1:5001/api/voos?aeroporto=SBGR&tipo=chegadas
```

Se aparecer JSON, deixe esse terminal aberto.

Terminal 2

Execute:

Shell

```
flutter run
```

Agora o aplicativo Flutter deverá buscar as informações do projeto Python.

Se o terminal do Python for fechado, o Flutter não conseguirá mais carregar os dados.

Desafio

Não existe apenas uma forma correta de construir a interface.

Utilizem os conhecimentos de Flutter vistos até agora para decidir:

- como os voos serão apresentados;
- quais informações aparecerão na tela inicial;
- quais informações aparecerão na tela de detalhes;
- como será feita a seleção entre chegadas, partidas e todos;
- como será feita a busca pelo aeroporto;
- como será o visual do aplicativo.

O mais importante é que o projeto apresente uma boa separação entre:

None

Service
Provider
Pages

e consiga realizar corretamente a comunicação:

None

Python → API → Flutter

Ao finalizar, o aplicativo deverá continuar funcionando com **os dois terminais abertos simultaneamente**: um executando o backend Python e outro executando o projeto Flutter.